

SECURITY AUDIT

Lista Dao

Substaker

FINAL

REPORT

Sep 2026

BAILSEC.IO



Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – Substaker – Audit Report
Website	listadao.org
Language	Solidity
Methods	Manual Analysis
Github repository	initial audited commit [reso1]: https://github.com/lista-dao/synclub-contracts/tree/ad380197fe4ad204f5c5f048f100783043d5190f new commit: https://github.com/lista-dao/synclub-contracts/tree/c0b075b04bfc9867188e4b0a32219236443fa254
Resolution 1	https://github.com/lista-dao/synclub-contracts/tree/f1e9d3648edc36b20de9b7bd689a30d4f90236bd

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	7			7		
Low	11	3		8		
Informational	5	1		4		
Governance						
Total	23	4		19		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

ListaStakeManager

The **ListaStakeManager** contract manages liquid staking for **slisBNB**. The current update makes four changes. It sends the stake for flagged validators to a **SubStaker** account in place of **StakeHub**, and it reads the balance of that account for those validators. It adds a caller whitelist to instant withdrawal. It moves the permission to unpause from the admin role to the manager role, and it adds **togglePause**. It adds two functions that accept a share count in place of a BNB amount.

Privileged Functions

- delegateTo
- redelegate
- redelegateShares
- undelegateFrom
- undelegateSharesFrom
- claimUndelegated
- instantWithdraw
- setInstantWhitelist
- setInstantWhitelistOff
- setSubStaker
- setSubValidator
- pause
- unpause
- togglePause

Invariants

- Each validator belongs to one account only.
- A validator holds no stake when its sub-validator flag changes.
- A redelegation uses the same account for the source and the destination validator.
- getTotalBnbInValidators counts each credit contract for one account only.

Issue_01	Queued head larger than staked BNB cannot be confirmed
Severity	Medium
Description	The protocol keeps some BNB liquid for instant withdrawals and stakes the rest with validators. A queued withdrawal is paid from BNB that has been unstaked and claimed back. A user can still request a withdrawal sized against everything [staked plus buffer]. Nothing stops a request bigger than what is actually staked. The bot can only unstake the validator shares. After the unbond period, that is all that comes back. To confirm the queue, the entire first unpaid request must fit in that returned pile. If it does not, confirmation does nothing. That first request stays unconfirmed, and everyone behind it is stuck too. The leftover buffer BNB is sitting right there, but the queue is not allowed to use it. While the buffer policy is on, the bot also cannot stake that pad just to unstake it later and close the gap. Instant withdrawals still work, but the slow queue does not. Example 95 BNB is staked, and 5 BNB sits in the buffer (100 BNB in the pool). Someone queues 97 BNB. The bot unstakes 95. <code>claimUndelegated</code> then adds that 95 to <code>undelegatedQuota</code>. The outer queue check still passes, so it sets <code>startIndex</code> to the first unconfirmed row [the 97]. <code>binarySearchCoveredMaxIndex[undelegatedQuota]</code> cannot walk past that row, so <code>coveredMaxIndex</code> stays at <code>startIndex</code>. <code>totalAmount</code> is then just that row's <code>amount</code>, which is 97. The inner check is if <code>{totalAmount <= undelegatedQuota}</code>. 97 is not $\leq$ 95, so that block is never entered: <code>undelegatedQuota</code> is not reduced, <code>nextConfirmedRequestUUID</code> is not advanced, and nothing is burned. The 5 in the buffer is unused. Later queued users wait behind that head until

	an admin shrinks the buffer, or enough new deposits sit above the buffer target, and another unbond cycle runs.
Recommendations	In <code>claimUndelegated</code> , if <code>undelegatedQuota</code> cannot cover the first unpaid row, take only the missing amount from <code>amountToDelegate</code> . Keep paying each row in full or not at all. Reduce <code>undelegatedQuota</code> and <code>totalDelegated</code> by the unstaked part, and <code>amountToDelegate</code> by the buffer part only.
Comments / Resolution	Acknowledged.

Issue_02	First-cycle cover reverts when <code>nextConfirmedRequestUUID</code> is zero
Severity	Medium
Description	<code>ListaStakeManager</code> uses <code>nextConfirmedRequestUUID</code> as a cursor over the withdrawal queue. A queued exit is claimable only after this cursor has moved past its uuid. <code>initialize</code> never sets the cursor, so it starts at 0. <code>requestWithdraw</code> increments <code>requestUUID</code> before assigning it, so the first exit is always uuid 1. After the bot pulls unbonded BNB from <code>StakeHub</code>, <code>claimUndelegated</code> asks <code>binarySearchCoveredMaxIndex</code> how far the quota covers. That helper requires the first queued uuid to be less than or equal to the cursor. On a fresh instance that check is $1 \leq 0$ and reverts. The <code>StakeHub</code> pull is in the same transaction, so it rolls back, and the cursor never advances. Undelegation still works, because <code>amountToUndelegate</code> treats a zero cursor as the start of the queue. Confirmation is the step that is blocked.
Recommendations	Set <code>nextConfirmedRequestUUID</code> to 1 in <code>initialize</code> , matching the first uuid that <code>requestWithdraw</code> assigns. Alternatively, treat a zero cursor as 1 in the cover search.
Comments / Resolution	Acknowledged.

Issue_03	Slash loss is not socialized; last withdrawers are stuck under an optimistic rate
Severity	Medium
Description	<code>compoundRewards</code> is the only way validator BNB is written into <code>totalDelegated</code>, which sets the <code>slisBNB/BNB</code> rate. It runs only when on-chain stake plus <code>undelegatedQuota</code> is strictly above the book; then it adds the gap. Nothing ever lowers <code>totalDelegated</code>. After a slash (or any shortfall), that check reverts, compounding stays stuck, and the rate keeps using the old, higher book. Withdrawals and undelegation are quoted as if the missing BNB were still there. Lista cannot pull that extra BNB from <code>StakeHub</code>. Early exits in the queue are still covered at the full quoted size from whatever <code>StakeHub</code> actually returns. The slash is therefore not shared across holders: it lands on whoever is left when the real balance runs out. Those last requests never become claimable, and the bot cannot finish undelegation for the quoted remainder – a DoS on the tail of the queue, not a freeze of the whole pool.
Recommendations	When validator BNB plus <code>undelegatedQuota</code> is below <code>totalDelegated</code> , write the book down to that sum so the rate absorbs the slash for every holder. Until that exists, do not cover queued withdrawals at the pre-slash quote.
Comments / Resolution	Acknowledged.

Issue_04	Refund burns stall without staking profit
Severity	Medium
Description	Lista refunds validator commission by minting manager-held slisBNB against BNB already added to the pool, then burning a daily slice so remaining holders gain the value over time. That burn runs only from compoundRewards, and compounding first requires validator BNB plus undelegation quota to exceed totalDelegated. At zero profit or a deficit (for example, redelegation-fee drift), the call reverts and a due burn never runs. Profit and the refund are separate: commission can already be scheduled while staking has nothing to compound. BOT retries do not move the schedule. When compounding later succeeds, only one daily amount is burned, and lastBurnTime is set, so skipped days are not caught up. Refunded value stays encoded in manager shares longer than intended. Users who unstake or transfer out during the stall miss the exchange-rate lift they would have received on the planned burn days. Example Manager refunds 30 BNB over 30 days (1 BNB of slisBNB per day). For 10 days there is no net staking profit, so every compoundRewards reverts and nothing is burned. On day 11 a small profit lets compounding succeed; only one daily slice burns, leaving 29/30 still sitting on the manager. A depositor who fully exits on day 10 leaves with none of that remaining refund. Completing the original 30-day plan then takes another 29 successful compound days after the stall, not 20.
Recommendations	Burn due refund slisBNB on its own authorized path, or burn a due daily amount before the profit check so the schedule can advance when there is no profit, without changing how profit and fees are accounted.
Comments / Resolution	Acknowledged.

Issue_05	Unbooked StakeHub redelegation fee overstates pooled BNB
Severity	Medium
Description	When the bot moves stake from validator A to validator B, BSC charges a small fee out of the money being moved, but Lista's books never subtract it, so the pool thinks it holds more BNB than it really does. The contract even ships the exact formula for the fee, <code>getRedelegateFee</code>, and nothing calls it. So the total delegated amount gradually drifts from the actual delegated amount.
Recommendations	Incorporate the redelegation drift fee.
Comments / Resolution	Acknowledged.

Issue_06	Instant-lane round trip captures each compound rate step
Severity	Medium
Description	Rewards are added to the books in one lump rather than continuously, so the price jumps at a single instant. Thus, an actor can deposit just before the jump and leave right after, capturing a major part of the rewards. <pre>uint256 totalProfit = totalBNBInValidators + undelegatedQuota - totalDelegated; // a whole window ... totalDelegated += totalProfit; // <-- one discrete step, not a smooth drip</pre> This can be done profitably if <code>instantWithdrawFeeRate</code> is 0.
Recommendations	Consider always implementing a withdrawal fee or acknowledge this issue.
Comments / Resolution	Acknowledged.

Issue_07	Re-routed validator tranche idles in quota, stalling confirmations
Severity	Medium
Description	Moving a validator between the main account and the SubStaker forces you to empty it first. The emptied BNB lands in a bucket that cannot be re-staked, and while it sits there, it also blocks the withdrawal queue from moving. <pre> if [credit.balanceOf(holder) != 0 credit.lockedBNBs(holder, 0) != 0] { revert ErrorsLib.AmountTooLarge(); // must be fully drained first } ... undelegatedQuota += undelegatedAmount; </pre>
Recommendations	Let delegateTo draw on surplus undelegatedQuota once the queue is settled, so a drained tranche can be put back to work.
Comments / Resolution	Acknowledged.

Issue_08	Leftover <code>undelegatedQuota</code> cannot confirm new requests without a <code>StakeHub</code> pull
Severity	Low
Description	<code>ListaStakeManager</code> only advances <code>nextConfirmedRequestUUID</code> inside <code>claimUndelegated</code>, and only after a <code>StakeHub</code> claim that increases the manager's balance. Extra BNB from a reserve-padded (or otherwise oversized) undelegate is kept in <code>undelegatedQuota</code>. <code>getAmountToUndelegate</code> subtracts that leftover, so later unconfirmed requests that fit inside it return 0 even though those rows are still unconfirmed. An honest bot then has nothing to undelegate. <code>claimUndelegated</code> reverts with "Nothing to claim" because there is no matured unbond to pull. Covering never runs, so those users cannot claim, even though the BNB is already on the manager, creating a liveness gap. If <code>reserveAmount</code> is above zero, the bot can still dummy-undelegate up to that cap and wait another 7 days. If <code>reserveAmount</code> is 0, there is no dummy path. The stall lasts until later demand exceeds the leftover; those users then still wait a full unbonding period for a pull they did not need. If demand never exceeds the leftover, they wait indefinitely. Example / Attack path 1. Users queue 10 BNB of exits. The bot undelegates 11 BNB (10 plus 1 BNB reserve padding). After unbonding, <code>claimUndelegated</code> covers the 10 BNB and leaves leftover quota $L = 1$ BNB. 2. A user queues a new 1 BNB exit. Unconfirmed prefix is $1 \text{ BNB} \leq L$, so <code>getAmountToUndelegate()</code> is 0. 3. The honest bot does nothing. <code>claimUndelegated</code> reverts "Nothing to claim". The request stays unconfirmed. 4. With <code>reserveAmount = 1 BNB</code>, the bot can dummy-undelegate 1 BNB, wait 7 more days, then the claim applies the leftover. With <code>reserveAmount = 0</code>, the stall lasts until further exits push unconfirmed demand above 1 BNB, and those users still wait 7 days after that extra undelegate.

Recommendations	Let the bot apply existing <code>undelegatedQuota</code> to unconfirmed requests without requiring a new StakeHub pull. Keep the covering logic, but expose a path that advances <code>nextConfirmedRequestUUID</code> when leftover quota already covers the prefix.
Comments / Resolution	Acknowledged.

Issue_09	BOT can repeatedly over-undelegate by <code>reserveAmount</code>
Severity	Low
Description	<code>undelegateFrom</code> and <code>undelegateSharesFrom</code> cap each call at <code>getAmountToUndelegate + reserveAmount</code>. <code>reserveAmount</code> is an admin-set rounding pad so the bot can slightly overshoot when share conversion floors and still cover the last withdrawal. It is never consumed. <code>getAmountToUndelegate</code> is unpaid queue minus BNB already in flight. When the queue is empty, or in-flight BNB already covers it, that value is 0, and the cap is just <code>reserveAmount</code> on every call. Each undelegate still increases <code>unbondingBnb</code>, but that does not shrink the pad. StakeHub has no 1 BNB floor on undelegate, so any $R > 0$ works. A broken bot, or a keeper that always calls <code>undelegateFrom</code>, can unbond stake not owed to the queue. Funds are not stolen: after 7 days they return as <code>undelegatedQuota</code>, and <code>totalDelegated</code> only falls when requests are covered. That BNB sits idle and stops earning yield. Default <code>reserveAmount</code> is 0. If <code>reserveAmount</code> is dust, draining TVL is theoretical, but if it is ~1 BNB as in the new tests, it is practical. Example / Attack Path Queue empty, <code>reserveAmount = 1 BNB</code>, 100 BNB delegated. Bot calls <code>undelegateFrom[validator, 1 BNB]</code>. Cap is $0 + 1$, call succeeds, <code>unbondingBnb</code> is 1 BNB. The next call still has cap 1 BNB. Repeating 100 times unbonds the whole position. Same after a real queue is already counted in <code>unbondingBnb</code>: a 10 BNB queue undelegated as 11 BNB leaves <code>getAmountToUndelegate() = 0</code>, and later calls may still move another 1 BNB each.
Recommendations	Limit extra undelegation to <code>reserveAmount</code> in total, not per call. Alternatively, allow the pad only when <code>getAmountToUndelegate > 0</code> .
Comments / Resolution	Acknowledged.

Issue_10	Delegation rejected when leftover buffer equals the max
Severity	Low
Description	<code>delegateTo</code> only stakes overflow above the configured buffer. After a stake, leftover BNB in <code>amountToDelegate</code> is allowed to sit at the max buffer, that is the intended cushion. <code>skipDelegateOrNot</code> treats leftover equal to the max the same as leftover below the max, so the bot is told to skip. A stake that would leave exactly the required buffer therefore reverts with <code>BufferTooSmall</code>, even though the buffer rule is already met. The practical effect is that the bot can never trim the cushion down to the target. It must leave at least a little extra unstaked. Combined with <code>StakeHub</code>'s minimum stake (~1 BNB), the stuck case is worse: if overflow is exactly that minimum, every legal amount is rejected, and that BNB stays in <code>amountToDelegate</code> until more deposits arrive. Funds are not stolen; they just are not staked. Example: max buffer is 100 BNB and 101 BNB is waiting (1 BNB overflow). Staking 1 BNB would leave 100, which matches the target, but the <code><=</code> check skips it. Staking less than 1 BNB fails <code>StakeHub</code>'s floor. That 1 BNB cannot be delegated. If 144 BNB is waiting instead, staking the full 44 BNB overflow is also skipped (leftover would be 100); the bot can only stake 43 BNB and leave 101.
Recommendations	In <code>skipDelegateOrNot</code> , skip only when the leftover buffer would be below the max, not when it is equal. Change <code>newBufferSize <= maxBufferSize</code> to <code>newBufferSize < maxBufferSize</code> .
Comments / Resolution	Acknowledged.

Issue_11	Instant-withdraw whitelist cannot bind the BNB beneficiary
Severity	Low
Description	<code>instantWithdraw</code> is gated by <code>instantWhitelist</code> (or by <code>instantWhitelistOff</code> when the list is globally disabled). The check is only on <code>msg.sender</code>. <code>slisBNB</code> is pulled from that caller, and BNB is sent back to the same address. The contract never records or validates the original shareholder or the final BNB recipient. That makes the list a trusted-caller / gateway control, not an end-user eligibility control. <code>slisBNB</code> is freely transferable. Anyone who is not listed can send shares to a listed address, have that address redeem, and receive the BNB. The unlisted holder never calls <code>instantWithdraw</code>, so the whitelist never sees them. The same gap exists if a listed router, wrapper, or helper is permissionless: anyone can route through it. The protocol cannot use this list for sanctions or beneficiary policy against users who can cooperate with a listed address.
Recommendations	Document the whitelist as a caller/gateway allowlist, not as user or sanctions enforcement. Do not list permissionless routers or wrappers. If the protocol needs on-chain beneficiary policy, bind permission to the beneficiary (for example signed permits with quotas) instead of checking only <code>msg.sender</code>.
Comments / Resolution	Acknowledged.

Issue_12	Validator removal leaves SubStaker routing in place
Severity	Low
Description	Removing a validator is meant to fully drop it from the pool: it must already be disabled, with no remaining delegated or unbonding BNB for that operator, after which the manager forgets the validator and its credit contract. The SubStaker flag is a separate map. Removal never clears it, and re-whitelisting only marks the operator active again and re-syncs credits. From then on, every delegate, undelegate, redelegate, and claim for that operator is sent through the SubStaker account instead of the manager, without any extra confirmation. That is easy to miss. Ops can believe the validator is a fresh manager-held position while _entry / _holder still treat it as a SubStaker tranche. The damage is operational: new stake lands on the wrong identity, voting power sits on SubStaker's govBNB instead of the manager's, and a later attempt to treat the validator as manager-owned (or to empty SubStaker so it can be rebound) will fail until someone notices the leftover flag.
Recommendations	On validator removal, also clear the SubStaker flag for that operator so a later whitelist starts as a manager-held position. Alternatively, require an explicit SubStaker assignment when re-whitelisting, and reject setSubValidator toward SubStaker while the SubStaker address is unset.
Comments / Resolution	Fixed. removeValidator now deletes subValidators[v] .

Issue_13	Pending <code>togglePause</code> can undo an intervening guardian pause
Severity	Low
Description	<code>ListaStakeManager</code> still exposes <code>togglePause</code> for <code>DEFAULT_ADMIN_ROLE</code>. That call does not take a target state. At execution, it only reads the live pause flag and flips it: paused becomes unpaused, unpaused becomes paused. Guardians already have a one-way <code>pause</code>, and managers have a one-way <code>unpause</code>. Those paths encode intent. <code>togglePause</code> does not. The admin role is also the slow path (timelock / queued ops). A queued flip is therefore bound to whatever the flag happens to be later, not to the state the admin meant to set. An admin transaction that was meant to pause can become an unpause if a guardian contains an incident first. The protocol then resumes while the incident is still open, until the guardian notices and pauses again. No attacker privilege is required beyond the honest race between admin tooling and emergency pause.
Recommendations	Stop using <code>togglePause</code> in operational tooling. Drive pause and resume only through <code>pause</code> and <code>unpause</code> . If the function must stay for compatibility, require an expected current state or an explicit target (pause vs unpause) and revert when the live flag does not match. Prefer removing it once callers have migrated.
Comments / Resolution	Acknowledged.

Issue_14	A 1-wei donation can delay SubStaker replacement
Severity	Low
Description	Replacing a SubStaker is a timelock-only action. Before the new account is bound, the old one must be empty: no StakeCredit shares, no BNB in the unbond queue, and a native balance of exactly zero. That last check is there so leftover BNB is not stranded when the manager stops routing to the old account. Native BNB on the SubStaker is not gated. Anyone can send it 1 wei through receive, and the public sweep function is the only way to send that balance back to the manager. Binding does not sweep first. If sweep and setSubStaker run in two transactions, the zero-balance check can fail even after the position and unbond queue are already drained. This can delays a migration off an old or faulty SubStaker. A failed bind can be retried, and each donation is recovered on the next sweep. The window closes if the timelock sweeps and binds in one transaction, but that path is not enforced on-chain.
Recommendations	Have setSubStaker sweep the old SubStaker before the drain checks, or require a single timelock batch that sweeps then binds. Keep the share and locked-BNB checks as they are.
Comments / Resolution	Fixed following recommendation.

Issue_15	Manager pause does not freeze SubStaker voting
Severity	Low
Description	Lista splits staking and govBNB voting across two identities: ListaStakeManager holds one tranche, and SubStaker holds the validators marked subValidator. Emergency pause is meant to freeze protocol movement, including vote delegation. Guardian pause on the manager does that for the manager's own govBNB: delegateVoteTo is whenNotPaused and onlyRole(DEFAULT_ADMIN_ROLE), so while the manager is paused, the admin cannot retarget that tranche. The SubStaker path is not part of that freeze. setVoteDelegatee is called on the SubStaker itself, not through a pause-gated manager function. It only checks that msg.sender currently holds DEFAULT_ADMIN_ROLE on the manager (hasRole(0x00, ...)). It never reads paused. Stake, undelegate, and redelegate on the SubStaker stay onlyStakeManager, so those still stop when the manager is paused. Vote movement does not, because it is a second privileged entrypoint that bypasses the manager entirely. The gap is operational: pause is inconsistent across the two holder identities. During an incident, operators may believe both vote buckets are frozen. They are not. Any current default admin can still point the SubStaker's entire govBNB balance at a new address, while the manager tranche stays locked until unpaused.
Recommendations	Have setVoteDelegatee revert when the manager is paused.
Comments / Resolution	Fixed following recommendation.

Issue_16	Refund floor leftover needs an extra day, and exact burns leave a stale daily amount
Severity	Low
Description	<code>refundCommission</code> splits minted <code>slisBNB</code> into <code>dailySlisBnb = total / days</code>. Integer division drops the remainder. Each <code>compoundRewards</code> burns once per day in <code>_burnRefundSlisBnb</code>. The last-day path only runs when <code>daily > remaining</code>, so an exact last burn zeros remaining by subtraction and leaves <code>dailySlisBnb</code> set. After the requested number of days, a leftover smaller than the daily amount can still be due, so the schedule needs one more burn than <code>_days</code>. When the total divides evenly, remaining hits zero and daily stays nonzero. Later burns skip because remaining is already zero, so public state still looks like a live daily burn. Example 10 <code>slisBNB</code> over 3 days: daily is $10 / 3 = 3$. After three burns, 1 remains. A fourth day burns it. The event said 3 days; completion took 4. 9 <code>slisBNB</code> over 3 days: daily is 3. Burns go $9 \rightarrow 6 \rightarrow 3 \rightarrow 0$. On the last day 3 is not greater than 3, so remaining is 0 and daily stays 3.
Recommendations	Fold in the division remainder, so the requested number of burns finishes the full amount. Alternatively, acknowledge and documents this edge case since its impact is negligible.
Comments / Resolution	Acknowledged.

Issue_17	Instant-withdraw buffer cannot be refilled from stake
Severity	Low
Description	Instant withdrawals are paid only from <code>amountToDelegate</code>, the idle BNB sitting in <code>ListaStakeManager</code> before the bot delegates it. <code>delegateTo</code> is meant to leave a slice of that idle BNB as a buffer (<code>bufferSizePct</code> of total pooled BNB) so instant withdrawals can be filled without waiting for the 7-day unbond. Instant withdrawals then spend that slice. Once the slice is gone, there is no path to pull BNB back from validators into the buffer. <code>undelegateFrom</code> / <code>undelegateSharesFrom</code> may undelegate at most <code>getAmountToUndelegate</code> plus <code>reserveAmount</code>. <code>getAmountToUndelegate</code> is only the unpaid queued <code>requestWithdraw</code> amount, so it never includes the drained instant-withdraw buffer. Even if the bot undelegates a bit extra via <code>reserveAmount</code>, the leftover is booked as <code>undelegatedQuota</code> and is not added to <code>amountToDelegate</code>. The only ways <code>amountToDelegate</code> grows again are new deposit calls and <code>refundCommission</code>. After the buffer is used up, further <code>instantWithdraw</code> calls revert until new deposits arrive.
Recommendations	Add a bot path that undelegates (or redirects leftover <code>undelegatedQuota</code>) specifically to restore <code>amountToDelegate</code> up to the target buffer, independent of the withdrawal queue. Until then, treat instant withdraw as best-effort against recent undeposited BNB, not a maintained percentage of TVL.
Comments / Resolution	Acknowledged.

Issue_18	<code>setSubValidator</code> can sub-flag a validator while <code>subStaker</code> is unbound, reverting all BOT operations for that validator
Severity	Informational
Description	<code>setSubValidator(_validator, true)</code> has no <code>subStaker != address[0]</code> guard. In the window between the manager upgrade and the timelock's <code>setSubStaker</code> binding, the admin can flag a validator as sub; every routed BOT operation (<code>delegateTo</code> , <code>undelegateFrom/undelegateSharesFrom</code> , <code>redelegate*</code> , <code>claimUndelegated</code>) for that validator then reverts until the flag is removed or the <code>SubStaker</code> is bound.
Recommendations	Either prevent early binding of validators or add a view helper to the bot to prevent delegating to such validators.
Comments / Resolution	Fixed by preventing <code>subValidator</code> setting if <code>subStaker</code> is 0.

Issue_19	New deposits are not used to fill the withdrawal queue
Severity	Informational
Description	Queued withdrawals (<code>requestWithdraw</code>) are filled only by undelegation on <code>StakeHub</code>. After the 7-day unbond, the bot calls <code>claimUndelegated</code>, which confirms request UUIDs. Users then claim. New deposits sit in <code>amountToDelegate</code> until <code>delegateTo</code>. <code>skipDelegateOrNot</code> only keeps <code>bufferSizePct</code> of pooled BNB for instant withdrawals. Anything above that is delegated. Neither step looks at <code>getAmountToUndelegate</code>. There is no path that pays the queue from idle deposits after the instant-withdraw buffer is reserved. The bot can therefore delegate fresh BNB and undelegate a similar amount of already-staked BNB at the same time. Queue users still wait the full unbond. The unbonding BNB earns nothing for those 7 days, while the new deposits could have paid the queue immediately and left the staked BNB in place.
Recommendations	After keeping the instant-withdraw buffer, consider using leftover <code>amountToDelegate</code> to cover the queue before delegating or undelegating. Reduce <code>amountToDelegate</code>, burn the matching <code>slisBNB</code>, and advance <code>nextConfirmedRequestUUID</code>. Do not reduce <code>totalDelegated</code> on this path. Undelegate from <code>StakeHub</code> only for the remainder the buffer cannot cover.
Comments / Resolution	Acknowledged.

Issue_20	Refund burns follow UTC dates, not 24 hours
Severity	Informational
Description	Commission refunds mint slisBNB to the stake manager and burn it in daily slices so the exchange-rate gain is spread over <code>_days</code>. Each successful <code>compoundRewards</code> call tries to burn one slice. The guard does not ask whether 24 hours have passed. It compares UTC calendar dates by dividing timestamps by 1 day. A burn is allowed as soon as today's date number is higher than the last burn's date. That matches the "burn once a day" comment, but two burns can be seconds apart if they sit on different UTC dates. The second compound still needs a little new staking profit, because the first call already booked the previous rewards. Around midnight that can be a tiny amount. If the call succeeds, a full daily slice is burned anyway. Total refund does not increase; the schedule just compresses. Anyone who deposits just before that pair can pick up two days of rate increase in one short window. The extra jump is one extra day's slice versus TVL.
Recommendations	Require <code>block.timestamp >= lastBurnTime + 1 days</code> (or store the next eligible timestamp) so burns are separated by a full day of elapsed time, not by a UTC date change.
Comments / Resolution	Acknowledged.

Issue_21	Removing a validator can hide SubStaker dust from rebind checks
Severity	Informational
Description	The pool proves a SubStaker is empty by walking creditContracts. removeValidator can drop a credit from that list while the SubStaker still holds leftover shares. removeValidator treats a validator as clear when getDelegated == 0 [pooled BNB plus locked BNB]. It does not check share balanceOf. Shares that convert to 0 BNB still pass, and syncCredits pops that credit. setSubStaker then calls requireBindable, which only inspects credits still on the list. Dust on a removed validator is never seen, so a new SubStaker can be bound while the old one still holds shares. After the bind, the manager routes to the new account and has no path to undelegate what remains on the old one. Those shares (and any BNB they later accrue) are stranded. This is not a large drain. It needs 0-BNB dust, a remove, then a SubStaker swap. setSubValidator already refuses that dust via balanceOf; the gap is that removal plus rebind does not.
Recommendations	Treat a validator as empty only when share balance and locked BNB are both zero, matching setSubValidator . When rebinding, do not rely only on the live creditContracts list; refuse the swap until the old SubStaker's shares and unbonding on every credit it used are zero.
Comments / Resolution	Acknowledged.

Issue_22	Bot batch claims can use stale withdrawal indices
Severity	Informational
Description	Each user has a list of pending withdrawals. <code>claimWithdraw</code> / <code>claimWithdrawFor</code> take an index into that list. After a successful claim, the contract copies the last request into the claimed slot and pops the tail (swap-and-pop). Later indices in the same list are no longer the requests they were before that claim. A user who claims one index, then reads the list again, is fine. A bot that loads several indices for the same user and submits them in one run is not: after the first claim, the remaining numbers point at different requests or past the end of the list. The batch reverts, or it pays a different request than the bot meant.
Recommendations	Document that <code>claimWithdrawFor</code> indices are only valid until the next claim for that user. Bot batches should claim from the highest index down, or refresh the list after each claim. Optionally claim by UUID instead of a moving array index.
Comments / Resolution	Acknowledged.



SubStaker

The **SubStaker** contract holds BNB stake for **ListaStakeManager** under a second account identity. **govBNB** permits one vote delegatee for each account, and **govBNB** and **StakeCredit** shares cannot move between accounts. A second account is therefore necessary to divide the pool vote power. The contract holds the stake for each validator that the manager marks as a sub validator. **ListaStakeManager** is the only permitted caller, and the manager is the only address that can receive BNB from this contract.

Privileged Functions

- initialize
- delegate
- undelegate
- redelegate
- claim
- setVoteDelegatee

Invariants

- claim sends only the BNB that StakeHub releases during that call.
- The vote power flag that goes to StakeHub is always false.
- Only TIMELOCK can approve an upgrade.

Issue_23	Hardcoded TIMELOCK leaves the SubStaker permanently un-upgradeable if governance rotates
Severity	Low
Description	TIMELOCK is correctly hardcoded to the actual timelock contract. However, if the governance addresses are ever to be rotated and the timelock contract changed, this contract will become unupgradable. The Substacker needs to be first redeployed to contain a new TIMELOCK constant, and only then can the previous timelock be retired.
Recommendations	Document this behaviour to make sure timelocks are not rotated without first upgrading this contract.
Comments / Resolution	Acknowledged.

SLisLibrary

The **SLisLibrary** library holds logic that **ListaStakeManager** calls by **delegatecall**. The update adds one guard function and moves two view functions into the library. The guard function **requireBindable** prevents a change of the **SubStaker** address until the old account holds no shares, no unbonding BNB, and no BNB balance. The two view functions calculate values for the withdrawal queue, and their logic did not change.

Invariants

- A SubStaker candidate must name the caller as its stake manager.
- A change of the SubStaker address requires the old account to be empty.
- **binarySearchCoveredMaxIndex** returns no index that costs more than the given BNB amount.